

A Generalization of the SAT Equation Theorem to Arbitrary CNF Formulas via Bitwise OR Encoding

Oscar Riveros
Santiago, RM, Chile
oscar.riveros@gmail.com

Abstract

The SAT Equation Theorem establishes that every balanced CNF formula over n Boolean variables is uniquely characterized by the integer $S = \sum_{C \in \mathcal{F}} 2^{\sum_{x \in C, x > 0} 2^{n-|x|}}$, whose binary expansion encodes the truth table of the formula: a 1 marks an unsatisfying assignment and a 0 marks a satisfying one. However, the original formulation is restricted to *balanced* clauses—those containing exactly one literal per variable—because the arithmetic sum introduces carries when multiple clauses share the same falsifying assignment.

In this paper we generalize the theorem to *arbitrary* CNF formulas (with clauses of any size) by replacing the arithmetic sum with the *bitwise OR* operation. Under this encoding, the contribution of each clause is a binary mask that sets a 1 in every position corresponding to an assignment that falsifies that clause. The OR of all such masks yields a number S whose binary expansion records, without carries, exactly the truth table of the entire formula. We provide a complete statement, proof, and illustrative examples of this generalization.

CCS Concepts: · Theory of computation $\rightarrow$ Logic.

Keywords: Satisfiability, SAT, CNF, Boolean functions, Number Theory, Truth-table encoding.

1 Introduction

In propositional logic, formulas written in Conjunctive Normal Form (CNF) are ubiquitous in automated reasoning, formal verification, and complexity theory. A CNF formula $\mathcal{F}$ over n Boolean variables $x_1, x_2, \dots, x_n$ is a conjunction of clauses, each being a disjunction of literals. A literal is either a positive occurrence $+i$ or a negative occurrence $-i$ of variable x_i .

A *balanced* CNF formula is one in which every clause contains exactly one literal per variable; i.e., each clause is a full disjunction of n literals, each over a distinct variable. For such formulas, Riveros [1] proved the following remarkable result:

Theorem 1.1 (SAT Equation Theorem—Balanced Case). *Let $\mathcal{F}$ be a balanced CNF formula over n Boolean variables. Define*

$$S = \sum_{C \in \mathcal{F}} 2^{\sum_{x \in C, x > 0} 2^{n-|x|}}.$$

If S is written in binary with at least 2^n digits, then the k -th bit is 1 if and only if the assignment encoded by the binary representation of k does not satisfy $\mathcal{F}$. Consequently, S uniquely encodes the truth table of $\mathcal{F}$.

The encoding is elegant and purely arithmetic; however, the theorem's original statement is limited to *balanced* clauses. When one attempts to extend the construction to clauses that omit some variables, two difficulties arise:

- (i) A clause missing t variables is falsified not by a single assignment but by an entire *subcube* of 2^t assignments. One must therefore encode a *set* of positions rather than a single position.
- (ii) Even if each clause is encoded as the sum of powers of two over its falsifying subcube, the arithmetic summation over *different* clauses may produce carries ($2^a + 2^a = 2^{a+1}$), breaking the one-to-one correspondence between bits and assignments.

In this paper we resolve both difficulties by replacing the arithmetic sum with the *bitwise OR* operation. This yields a clean, carry-free generalization that applies to **any** CNF formula, regardless of clause size or structure.

2 Preliminaries

We work over N Boolean variables $x_1, x_2, \dots, x_N$, each taking a value in $\{0, 1\}$. We adopt the convention that 0 represents **True** and 1 represents **False** when evaluating literals. A literal is $+i$ (positive) or $-i$ (negative). A clause C is a disjunction of literals; a CNF formula $\mathcal{F} = C_1 \wedge C_2 \wedge \dots \wedge C_m$ is a conjunction of clauses.

Definition 2.1 (Position of an assignment). *For an assignment $a = (a_1, a_2, \dots, a_N) \in \{0, 1\}^N$, its position is the integer*

$$\text{pos}(a) = \sum_{i=1}^N a_i \cdot 2^{N-i}.$$

Thus pos is a bijection between $\{0, 1\}^N$ and $\{0, 1, \dots, 2^N - 1\}$, interpreted as indices of bits in a 2^N -bit binary number.

Definition 2.2 (Present and absent variables in a clause). *For a clause C , let*

$$\begin{aligned} P_C &= \{i \in \{1, \dots, N\} \mid +i \in C \text{ or } -i \in C\} \quad (\text{variables present in } C), \\ M_C &= \{1, \dots, N\} \setminus P_C \quad (\text{variables absent from } C). \end{aligned}$$

Definition 2.3 (Required bit). *For each present variable $i \in P_C$, the required bit $r_i(C)$ is the value that makes the literal false:*

$$r_i(C) = \begin{cases} 0, & \text{if } +i \in C \quad (\text{since } +i \text{ is false when } x_i = 0), \\ 1, & \text{if } -i \in C \quad (\text{since } -i \text{ is false when } x_i = 1). \end{cases}$$

With these definitions, a clause C is false exactly when $x_i = r_i(C)$ for all $i \in P_C$. The absent variables M_C may take any value without affecting the falsity of C .

Definition 2.4 (Base contribution of a clause). *The base contribution $B(C)$ encodes the required bits of the present variables:*

$$B(C) = \sum_{i \in P_C} r_i(C) \cdot 2^{N-i}.$$

3 Encoding a Single Clause via OR

Let C be a clause and let $M_C = \{i_1, i_2, \dots, i_t\}$ be the set of its absent variables, where $t = |M_C| = N - |P_C|$. The set of all assignments that falsify C is precisely

$$\text{Fals}(C) = \{a \in \{0, 1\}^N \mid a_i = r_i(C) \ \forall i \in P_C\}.$$

For each subset $M' \subseteq M_C$, we obtain a distinct falsifying assignment by setting the variables in M' to 1 and the remaining absent variables to 0. The position of this assignment is

$$B(C) + \sum_{i \in M'} 2^{N-i}.$$

Therefore the contribution of clause C to the global encoding is the bitwise OR of 2^t distinct powers of two:

$$S_C = \bigvee_{M' \subseteq M_C} 2^{B(C) + \sum_{i \in M'} 2^{N-i}},$$

where $\bigvee$ denotes the bitwise OR operation. Equivalently, S_C is the binary number whose k -th bit is 1 if and only if $k = B(C) + \sum_{i \in M'} 2^{N-i}$ for some $M' \subseteq M_C$.

Remark 3.1. In the balanced case, $P_C = \{1, \dots, N\}$ and $M_C = \emptyset$. Then $S_C = 2^{B(C)} = 2^{T(C)}$, recovering the original encoding.

4 Encoding a CNF Formula

A CNF formula $\mathcal{F} = C_1 \wedge C_2 \wedge \dots \wedge C_m$ is falsified by an assignment a if and only if *at least one* clause C_j is falsified by a . Consequently, the set of all falsifying assignments is the *union* of the falsifying sets of the individual clauses.

Since each S_{C_j} has a 1 precisely in the positions corresponding to $\text{Fals}(C_j)$, the number encoding the union is simply the bitwise OR of the individual contributions:

$$S = \bigvee_{j=1}^m S_{C_j}.$$

Combining the two levels, we obtain the fully expanded form:

$$S = \bigvee_{j=1}^m \bigvee_{M' \subseteq M_{C_j}} 2^{B(C_j) + \sum_{i \in M'} 2^{N-i}}.$$

This is the central definition of this paper.

5 The Generalized SAT Equation Theorem

Theorem 5.1 (Generalized SAT Equation Theorem). *Let $\mathcal{F}$ be an arbitrary CNF formula over N Boolean variables, with m clauses of any sizes. Let S be defined as above via the bitwise OR encoding. Then, when S is written in binary with 2^N bits, the following holds for every assignment $a \in \{0, 1\}^N$:*

$$\text{the bit at position } \text{pos}(a) \text{ is } \begin{cases} 1, & \text{if } a \text{ does } \textbf{not} \text{ satisfy } \mathcal{F}, \\ 0, & \text{if } a \text{ satisfies } \mathcal{F}. \end{cases}$$

In particular, S is a unique and exact integer encoding of the truth table of $\mathcal{F}$.

Proof. The proof follows directly from the construction.

Encoding of a single clause. By definition, for each clause C_j ,

$$S_{C_j} = \bigvee_{M' \subseteq M_{C_j}} 2^{B(C_j) + \sum_{i \in M'} 2^{N-i}}.$$

All the exponents here are distinct because each subset M' produces a different sum of weights. Hence S_{C_j} is a well-defined binary number in which the k -th bit is 1 iff $k = \text{pos}(a)$ for some $a \in \text{Fals}(C_j)$. Thus S_{C_j} encodes exactly the set of assignments that falsify C_j .

Encoding of the whole formula. An assignment a satisfies $\mathcal{F}$ iff it satisfies *every* clause C_j , i.e., iff $a \notin \bigcup_{j=1}^m \text{Fals}(C_j)$. Equivalently, a falsifies $\mathcal{F}$ iff there exists j such that $a \in \text{Fals}(C_j)$.

Now,

$$S = \bigvee_{j=1}^m S_{C_j}.$$

Since OR corresponds to set union on bit-positions, the k -th bit of S is 1 iff there exists some j for which the k -th bit of S_{C_j} is 1, i.e., iff there exists some C_j and some $a \in \text{Fals}(C_j)$ with $\text{pos}(a) = k$. This is precisely the condition that a falsifies $\mathcal{F}$.

Carry-freeness. Because the OR operation is idempotent ($1 \vee 1 = 1$), if two different clauses (or two subsets M' of the same clause) set the same bit, the result is still 1. No carries are generated. The one-to-one correspondence between bit positions and assignments is thus preserved unconditionally.

Conclusion. The binary expansion of S has a 1 at position $\text{pos}(a)$ iff a is an unsatisfying assignment, and 0 otherwise. Hence S uniquely encodes the truth table of $\mathcal{F}$. $\square$

6 Illustrative Examples

6.1 Balanced case (recovering the original theorem)

Let $N = 3$ and $\mathcal{F} = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$.

- C_1 : $+x_1, +x_2, -x_3$. $P_{C_1} = \{1, 2, 3\}$, $M_{C_1} = \emptyset$.

$$r_1 = 0, r_2 = 0, r_3 = 1 \implies B(C_1) = 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 1.$$

$$S_{C_1} = 2^1 = 2 \text{ (binary 00000010, bit 1 is 1).}$$

- C_2 : $-x_1, +x_2, +x_3$.

$$r_1 = 1, r_2 = 0, r_3 = 0 \implies B(C_2) = 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 = 4.$$

$$S_{C_2} = 2^4 = 16 \text{ (binary 00010000, bit 4 is 1).}$$

$S = 2 \vee 16 = 18$ (binary 00010010). Bits 1 and 4 are 1, meaning assignments $(0, 0, 1)$ and $(1, 0, 0)$ falsify $\mathcal{F}$; all other 6 assignments satisfy $\mathcal{F}$. This matches the original theorem.

6.2 Non-balanced clause

Let $N = 3$ and $\mathcal{F}$ consist of a single clause $C = (x_1 \vee \neg x_2)$, with x_3 absent.

- $P_C = \{1, 2\}$, $M_C = \{3\}$.
- $r_1 = 0$ (since $+x_1$), $r_2 = 1$ (since $-x_2$).
- $B(C) = 0 \cdot 2^2 + 1 \cdot 2^1 = 2$.
- Weight of x_3 : $2^{3-3} = 2^0 = 1$.

Subsets of $M_C = \{3\}$:

$$\begin{aligned} M' = \emptyset : \quad 2^{B(C)+0} &= 2^2 = 4, \\ M' = \{3\} : \quad 2^{B(C)+1} &= 2^3 = 8. \end{aligned}$$

$$S = 4 \vee 8 = 12 \quad (\text{binary } 00001100).$$

Bits at positions 2 and 3 are 1:

- Position 2 = $(0, 1, 0)$: $x_1 = 0$, $x_2 = 1$. In C , $+x_1$ is false and $-x_2$ is false $\Rightarrow$ clause false. $\checkmark$
- Position 3 = $(0, 1, 1)$: same for x_1, x_2, x_3 arbitrary. Clause false. $\checkmark$

All other assignments satisfy C . This example illustrates how a single missing variable doubles the number of falsifying positions (a 1-dimensional subcube).

6.3 Two clauses sharing a falsifying assignment

Let $N = 3$,

$$\mathcal{F} = (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2 \vee x_3).$$

- C_1 : $P_{C_1} = \{1, 2\}$, $M_{C_1} = \{3\}$, $B(C_1) = 0 \cdot 4 + 1 \cdot 2 = 2$.

$$S_{C_1} = 2^2 \vee 2^{2+1} = 4 \vee 8 = 12.$$

- C_2 : $P_{C_2} = \{1, 2, 3\}$, $M_{C_2} = \emptyset$, $r_1 = 1$, $r_2 = 0$, $r_3 = 0$, $B(C_2) = 4$.

$$S_{C_2} = 2^4 = 16.$$

$$S = 12 \vee 16 = 28 \quad (\text{binary } 00011100).$$

Bits 2, 3, and 4 are 1. The assignment $(1, 0, 0)$ (position 4) falsifies only C_2 ; assignments $(0, 1, 0)$ and $(0, 1, 1)$ (positions 2 and 3) falsify C_1 . There is no overlap between clause contributions, but even if there were, the OR operation handles it without carries.

7 Discussion

7.1 Why OR is essential

The crucial advantage of the bitwise OR over the arithmetic sum is *idempotence*: $1 \vee 1 = 1$, whereas $2^a + 2^a = 2^{a+1}$ produces a carry that shifts the bit to a higher position, destroying the correspondence between bit indices and assignments. With OR, overlapping falsifying assignments from different clauses are handled implicitly and correctly.

7.2 Arithmetic alternative with base $b > m$

If one insists on using arithmetic addition for theoretical reasons, carries can be avoided by choosing a base b strictly larger than the maximum number of clauses that can share a given falsifying position. Since each assignment can be falsified by at most m clauses, using

$$S = \sum_{j=1}^m \sum_{M' \subseteq M_{C_j}} b^{B(C_j) + \sum_{i \in M'} 2^{N-i}}$$

with $b > m$ guarantees that no digit exceeds $b-1$ and no carries occur. However, this yields a base- b representation rather than a binary one, and is less natural than the OR formulation.

7.3 Recovering the satisfying assignments from S

Given S , one can test whether an assignment a satisfies $\mathcal{F}$ simply by inspecting the $\text{pos}(a)$ -th bit of S :

$$a \text{ satisfies } \mathcal{F} \iff \lfloor S / 2^{\text{pos}(a)} \rfloor \bmod 2 = 0.$$

To enumerate all satisfying assignments, one iterates over $k = 0, 1, \dots, 2^N - 1$ and collects those k for which the k -th bit of S is 0. This is a straightforward procedure.

7.4 Computational considerations

The integer S can be as large as $2^{2^N} - 1$, requiring 2^N bits to represent. While this is exponential in N , the theorem is a *structural* result about the Boolean function encoded by a CNF formula, not an algorithmic claim about efficient manipulation. It provides a compact *combinatorial* description of the truth table and may find applications in areas such as:

- Uniqueness proofs for SAT encodings.
- Algebraic characterizations of Boolean functions.
- Theoretical connections between number theory and propositional logic.

8 Conclusion

We have generalized the SAT Equation Theorem from balanced CNF formulas to *arbitrary* CNF formulas by replacing the arithmetic sum with the bitwise OR operation. The resulting construction produces a single integer S , defined by

$$S = \bigvee_{j=1}^m \bigvee_{M' \subseteq M_{C_j}} 2^{B(C_j) + \sum_{i \in M'} 2^{N-i}},$$

whose binary expansion of length 2^N encodes exactly the truth table of the formula: bits at positions corresponding to unsatisfying assignments are 1, and all other bits are 0. This encoding is unique, carry-free, and applies without restriction to any CNF formula over any number of variables.

The bitwise OR technique resolves the fundamental difficulty posed by non-balanced clauses—namely, that a clause missing t variables generates a t -dimensional subcube of falsifying assignments rather than a single one—and simultaneously eliminates the carry problem inherent in arithmetic summation. We believe this generalization opens new avenues for exploring connections between propositional satisfiability and number-theoretic representations of Boolean functions.

References

- [1] Oscar Riveros. *A Unique Encoding of Satisfying Assignments for Balanced CNFs*. Internet Publication Research 2025.